

Software Requirements Specification

for

Pocket Packet Sniffer

Version 1.3

Prepared by:

- Vikas Kulakarni
- Shivanand
- Shivakumara gp
- Sanjith R

Table of Contents

Pocket Packet Sniffer.....	1
Table of Contents.....	2
Revision History	4
1. Introduction	5
1.1 Purpose	5
1.2 Intended Audience and Reading Suggestions	5
1.3 Product Scope	5
1.4 References	5
2. Overall Description.....	6
2.1 Product Perspective	6
2.2 Product Functions	6
2.3 User Classes and Characteristics	6
2.4 Operating Environment	6
2.5 Design and Implementation Constraints	6
2.6 Assumptions and Dependencies	6
3. External Interface Requirements	7
3.1 User Interfaces	7
3.2 Software Interfaces	7
3.3 Communications Interfaces	7
3.4 Hardware Interfaces.....	7
4. Analysis Models	8
Use Case Diagram	8
Sequence Diagram: Capture to Alert.....	9
5. System Features.....	10
5.1 Packet Capture.....	10
Description and Priority.....	10
Stimulus/Response Sequences.....	10
Functional Requirements	10
5.2 Packet Parsing and Classification	10
Description and Priority.....	10
Stimulus/Response Sequences.....	10
Functional Requirements	10
5.3 Capture Storage and Query.....	10
Description and Priority.....	10
Stimulus/Response Sequences.....	10
Functional Requirements	10
5.4 Live Monitoring Dashboard.....	10

Description and Priority.....	10
Stimulus/Response Sequences.....	11
Functional Requirements	11
5.5 Suspicious Activity Detection (Stretch).....	11
Description and Priority.....	11
Stimulus/Response Sequences.....	11
Functional Requirements	11
5.6 Multi-Backend Capture Support (Stretch)	11
Description and Priority.....	11
Stimulus/Response Sequences.....	11
Functional Requirements	11
6. Other Nonfunctional Requirements	12
6.1 Performance Requirements.....	12
6.2 Safety Requirements.....	12
6.3 Security Requirements.....	12
6.4 Software Quality Attributes	12
6.5 Business Rules	12
7. Other Requirements	13
Appendix A: Glossary	13
Appendix B: Field Layouts.....	13
Appendix C: Requirement Traceability Matrix	14
Appendix D: Risks We're Watching.....	14

Revision History

Name	Date	Reason for Changes	Version
VIKAS KULAKARNI	01-09-2026	Initial draft covering project scope, purpose and overall description	1.0
SHIVANAND	02-09- 2026	Added external interface requirements and system features	1.1
SHIVAKUMARA GP	03-09-2026	Added analysis models: layered architecture and use case diagrams	1.2
SANJITH R	03-09-2026	Final review: non-functional requirements, formatting and consistency check	1.3

1. Introduction

1.1 Purpose

This document lays out what we're building with Pocket Packet Sniffer (v1.0) and why. It's our Software Engineering course project, so this SRS covers the whole thing — a small system split into a capture stage, an analysis stage, a storage stage, and a way of showing the results to the user.

1.2 Intended Audience and Reading Suggestions

Three groups will probably read this: us (the team, while we're building it), our instructor evaluating the design, and anyone who picks this project up later and wants to add a new capture source or analyzer. If you're new to the project, Sections 1–2 give context, Section 5 covers what the system actually does, and Section 6 covers the quality bar we're holding ourselves to. Section 4 is the fastest way to see how the pieces connect.

1.3 Product Scope

Pocket Packet Sniffer grabs network packets over WiFi, Ethernet, or BLE, figures out what they are, stores them, and shows a summary through a command-line tool (a web dashboard is a nice-to-have if time allows). Since this is a course project, we're deliberately weighting this toward good design — clean separation between parts, room to extend it, and code we can actually test — over cramming in every feature we can think of.

1.4 References

- Scapy documentation — <https://scapy.readthedocs.io>
- Zeek documentation — <https://docs.zeek.org>
- IEEE SRS template (this document follows its structure)
- libpcap / WinPcap packet capture library documentation

2. Overall Description

2.1 Product Perspective

This is a new project we're building from scratch — it isn't replacing anything or plugging into a bigger system. Section 4 has a diagram showing how the internal pieces are arranged and where the system touches the outside world (network interfaces, storage, and the user).

2.2 Product Functions

- Capture raw packets from a chosen network interface
- Parse and classify captured packets
- Persist captures for later inspection
- Filter and query stored captures
- Show live traffic statistics
- (Stretch) Flag suspicious traffic patterns
- (Stretch) Run more than one capture backend at once

2.3 User Classes and Characteristics

Evaluator/Instructor: grading design quality and code structure, doesn't need deep networking background but will expect the parts to be cleanly separated. Developer/Team Member: the person adding a new capture backend or analyzer later, so the interfaces between layers need to be documented well enough for that. End User: someone comfortable with a terminal who wants visibility into their own network without a steep learning curve.

2.4 Operating Environment

Target hardware: a Raspberry Pi to start (an ESP32-class device if we get to BLE capture). OS: Linux, most likely Raspberry Pi OS. Runtime: Python 3.x. Zeek is an optional add-on for deeper protocol analysis if we reach that stretch goal.

2.5 Design and Implementation Constraints

- Capture, analysis, storage, and presentation need to stay as separate, swappable pieces — this is a course requirement, not just something nice to have
- Has to run comfortably on Raspberry Pi / ESP32-class hardware, so we can't be careless about CPU or memory use
- Capture relies on whatever libraries are actually available on the target OS (Scapy, libpcap)
- We only capture traffic on networks/devices we're actually allowed to monitor

2.6 Assumptions and Dependencies

- We're assuming we'll have at least one Raspberry Pi (or similar) to build and demo on
- If we go with the WiFi backend, we're assuming access to an adapter that supports monitor mode
- We're relying on Scapy (and maybe Zeek) staying compatible with whatever OS/Python version we settle on
- BLE capture, if we get to it, assumes we can get our hands on a compatible BLE sniffer or ESP32 firmware

3. External Interface Requirements

3.1 User Interfaces

The main interface is a command-line tool — commands to start or stop a capture, list and filter past sessions, and print live stats (packet rate, protocol breakdown, top talkers) to the terminal. A web dashboard covering the same views is a stretch goal. Error messages should say plainly what went wrong (missing permissions, interface not found) and what to try next, instead of just dumping a stack trace.

3.2 Software Interfaces

- Scapy (Python) — does the actual packet capture and low-level parsing
- Zeek (optional, stretch) — for deeper protocol-level logs
- Storage — pcap files for raw captures, plus something like SQLite for indexed metadata, accessed through a repository-style interface so we can swap it later
- The OS's packet capture API (libpcap), used through Scapy

3.3 Communications Interfaces

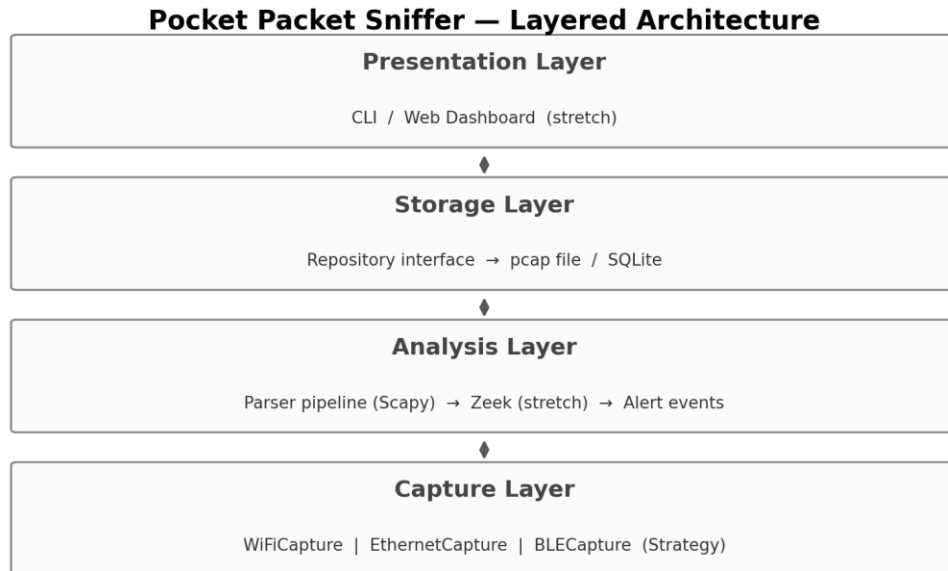
Nothing needs network access for the CLI version. If we build the web dashboard, it'll just serve over local HTTP — no calls out to any external service.

3.4 Hardware Interfaces

- A network interface that supports monitor/promiscuous mode, for WiFi capture
- A regular Ethernet NIC for wired capture
- (Stretch) A BLE-capable radio — an ESP32 or an nRF52840 dongle — for BLE advertisement scanning

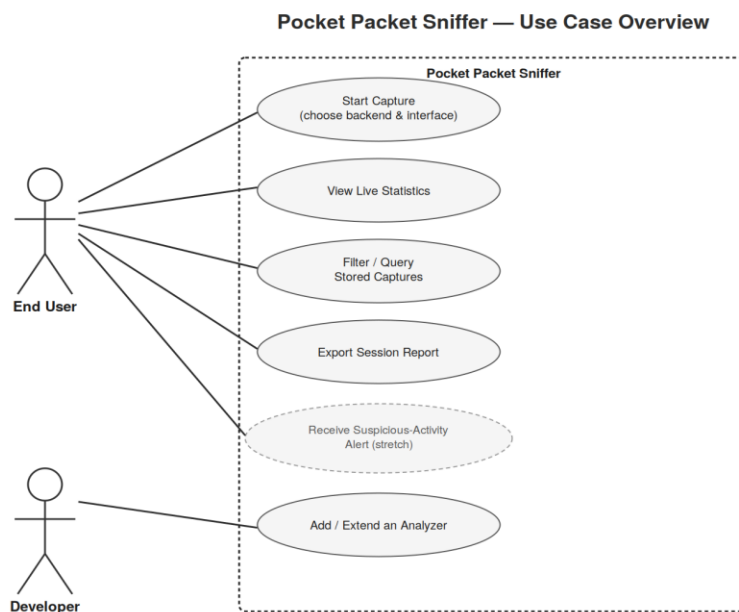
4. Analysis Models

The diagram below is our layered architecture. Each layer only talks to its neighbor through a defined interface — that's really the whole point of the Modularity and Extensibility requirements in Section 6.



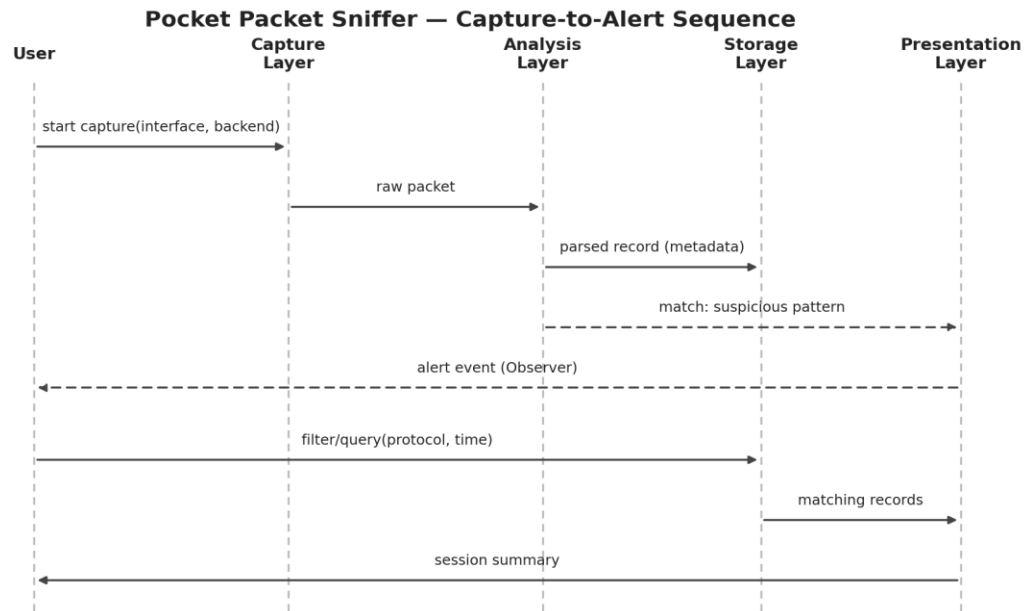
Use Case Diagram

This covers the same functions as the list in Section 2.2, but shown as actors and use cases so it's easier to see who triggers what.



Sequence Diagram: Capture to Alert

This walks through one full pass of the pipeline — a packet coming in, getting parsed, matching a suspicious pattern, and the resulting alert and a later query both making it back to the user. It's the clearest way to see why the layers need well-defined interfaces: every arrow here is a boundary that could later be swapped out (a different capture backend, a different storage engine) without touching the rest.



5. System Features

5.1 Packet Capture

Description and Priority

Priority: High.

Stimulus/Response Sequences

- User picks an interface and backend and starts a capture; the system begins streaming packets from that interface.

Functional Requirements

- REQ-1: The system shall capture raw packets from a user-selected interface (WiFi monitor mode, Ethernet, or BLE advertisement scanning).
- REQ-2: The system shall show a clear error if the chosen interface is unavailable or the process lacks permission to use it.

5.2 Packet Parsing and Classification

Description and Priority

Priority: High.

Stimulus/Response Sequences

- As packets come in from the Capture Layer, the Analysis Layer parses each one and pulls out the metadata we care about.

Functional Requirements

- REQ-3: The system shall extract source/destination address, protocol, size, and timestamp from each captured packet.
- REQ-4: A new protocol analyzer shall be addable without changing any existing analyzer (pipeline-style design).

5.3 Capture Storage and Query

Description and Priority

Priority: High.

Stimulus/Response Sequences

- A capture session gets saved (automatically or on request); later, the user filters or searches it by protocol, address, or time window.

Functional Requirements

- REQ-5: The system shall save captures as pcap files and/or an indexed metadata store.
- REQ-6: The system shall let the user filter or query stored captures by protocol, address, or time window.

5.4 Live Monitoring Dashboard

Description and Priority

Priority: Medium.

Stimulus/Response Sequences

- While a capture is running, the user can see live packet rate, protocol breakdown, and top talkers through the CLI (or the web dashboard, if we get there).

Functional Requirements

- REQ-7: The system shall show live packet rate, protocol breakdown, and the top 5 talkers during an active capture.
- REQ-8: The system shall let the user export a session summary as CSV or PDF.

5.5 Suspicious Activity Detection (Stretch)

Description and Priority

Priority: Low.

Stimulus/Response Sequences

- As traffic is analyzed, the system checks it against a handful of simple rules (ARP spoofing, a rogue AP, an unfamiliar device joining) and raises an event when something matches.

Functional Requirements

- REQ-9: Known suspicious patterns (ARP spoofing, rogue AP, unrecognized device) shall be flagged through an event that the Presentation Layer can subscribe to (Observer pattern).

5.6 Multi-Backend Capture Support (Stretch)

Description and Priority

Priority: Low.

Stimulus/Response Sequences

- User runs two capture backends at the same time (say, Ethernet and BLE) and sees combined stats.

Functional Requirements

- REQ-10: The system shall support running more than one capture backend at once without requiring changes to the Analysis, Storage, or Presentation layers.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

- The system should keep up with capturing and parsing typical home/lab network traffic on Raspberry Pi – class hardware without dropping packets.
- Live dashboard numbers should refresh at least every 2 seconds while a capture is running.

6.2 Safety Requirements

This is a passive capture tool only — it should never inject, alter, or replay traffic on the network it's watching. That's not something we want to make configurable; it should just always be true.

6.3 Security Requirements

- The tool should only ever be pointed at networks/devices we're actually authorized to monitor.
- Captures can contain sensitive stuff (credentials, personal traffic), so saved files need to be locked down with normal file permissions.
- Captured data shouldn't leave the device unless the user explicitly exports it — no silent uploads anywhere.

6.4 Software Quality Attributes

Attribute	What we mean by it here
Modularity	Capture, analysis, storage, and presentation stay as separate pieces behind clear interfaces, so any one of them can be swapped without touching the rest.
Extensibility	Adding a new protocol analyzer or capture backend shouldn't mean editing code that has nothing to do with it (Strategy pattern for backends, pipeline for analyzers).
Maintainability	Code organized by layer, with documented interfaces and unit tests on the parsers and core logic, so someone new can work on one layer without reading the whole codebase first.
Portability	Runs fine within the resource limits of a Raspberry Pi or ESP32-class device.
Reliability	If an interface or permission fails, the capture pipeline should fail gracefully, not crash the whole tool.
Usability	CLI/dashboard output should make sense without a manual — clear labels, sensible defaults, no hand-editing config files just to get started.

6.5 Business Rules

- Only the project team pushes changes to the main repository, and changes get reviewed before merging.
- Any third-party library we use (Scapy, Zeek, etc.) gets credited, per the course's academic integrity policy.

7. Other Requirements

- Legal: capture has to stay within local laws on network interception — we only monitor networks/devices we own or have explicit permission to touch.
- Reuse: open-source libraries (Scapy, Zeek) are fine to use as long as they're cited in the project report, per course policy.
- Internationalization: not a concern for this version — English-only output is fine.

Appendix A: Glossary

Term	What it means
pcap	A standard file format for storing captured network packets.
Promiscuous / Monitor mode	A network interface mode that picks up all traffic on a channel, not just traffic addressed to that device.
ARP spoofing	An attack where a device sends fake ARP messages to link its own address to another host's IP, letting it intercept traffic.
BLE advertisement	The periodic broadcast packets Bluetooth Low Energy devices send out, which can be picked up without pairing.
Zeek	An open-source network security monitoring tool that turns raw traffic into protocol-level logs.
Strategy pattern	A design pattern where a few interchangeable options (here, capture backends) share one common interface, and you pick one at runtime.
Observer pattern	A design pattern where interested parties subscribe to events from a source, instead of that source needing to know about them directly.

Appendix B: Field Layouts

Fields we capture and store for each packet record:

Field	Data Type	Description	Mandatory
Timestamp	DateTime	Time the packet was captured	Y
Source Address	String	Source IP or MAC address	Y
Destination Address	String	Destination IP or MAC address	Y
Protocol	String	Identified protocol (e.g. TCP, UDP, ARP, BLE-ADV)	Y
Packet Size	Integer	Size of the packet in bytes	Y
Interface	String	Which capture backend/interface it came in on	Y
Flags/Notes	String	Optional analyzer-generated flags (e.g. matched a suspicious pattern)	N

Appendix C: Requirement Traceability Matrix

We'll fill in the Code File and Test Case columns as we actually build each piece — for now this maps every requirement to where it lives architecturally, so nothing gets forgotten once implementation starts.

Req ID	Brief Description	Architecture Reference	Design Reference	Code File	Test Case ID
REQ-1	Capture raw packets	Capture Layer	Capturer interface	TBD	TBD
REQ-2	Report interface/permission errors	Capture Layer	Capturer interface	TBD	TBD
REQ-3	Extract packet metadata	Analysis Layer	Parser pipeline	TBD	TBD
REQ-4	Add analyzers without touching others	Analysis Layer	Parser pipeline	TBD	TBD
REQ-5	Persist captures	Storage Layer	Repository interface	TBD	TBD
REQ-6	Filter/query stored captures	Storage Layer	Repository interface	TBD	TBD
REQ-7	Live stats display	Presentation Layer	CLI dashboard module	TBD	TBD
REQ-8	Export session report	Presentation Layer	Report exporter	TBD	TBD
REQ-9	Suspicious activity alert	Analysis → Presentation	Observer/event bus	TBD	TBD
REQ-10	Concurrent multi-backend capture	Capture Layer	Capturer interface (multiple instances)	TBD	TBD

Appendix D: Risks We're Watching

Risk	Why It Matters	What We'll Do About It
Monitor-mode WiFi adapter isn't available or doesn't work on the day	The WiFi backend becomes a blocker for the MVP demo	Start with the Ethernet backend, which needs no special hardware, and treat WiFi as an add-on
BLE/ESP32 stretch goal takes longer than expected	Could eat into time meant for polishing the MVP	Keep it explicitly a stretch goal — MVP demo doesn't depend on it going well
Layer boundaries get blurry under deadline pressure (e.g. Analysis code reaching into Storage directly)	Undermines the whole point of the design — this is what's being graded	Review interfaces before writing implementation code; treat a boundary violation as a bug, not a shortcut
Captured traffic includes sensitive data we didn't expect	Privacy/ethics concern even in a course setting	Only capture on networks we own or have explicit permission for; keep captures local and access-controlled